

Revisão de POO

Programação Orientada a Objetos

Conceitos fundamentais de POO: classes, objetos, herança, polimorfismo e encapsulamento aplicados de forma prática.

Introdução à POO

A base da programação
moderna

//

*A Programação Orientada a Objetos
não é apenas sobre escrever código. É
sobre como modelar o mundo real
dentro do computador*

//

Ramalho, 2023.

O que é POO?



Paradigma

É uma forma de pensar e organizar o código. Em vez de focar apenas em funções, focamos em "objetos" que combinam dados e comportamentos.



Reutilização

Permite criar blocos de código que podem ser reaproveitados em diferentes partes do sistema, economizando tempo e esforço.



Manutenção

Sistemas organizados em objetos são muito mais fáceis de entender, atualizar e corrigir bugs no futuro.

1. Classes

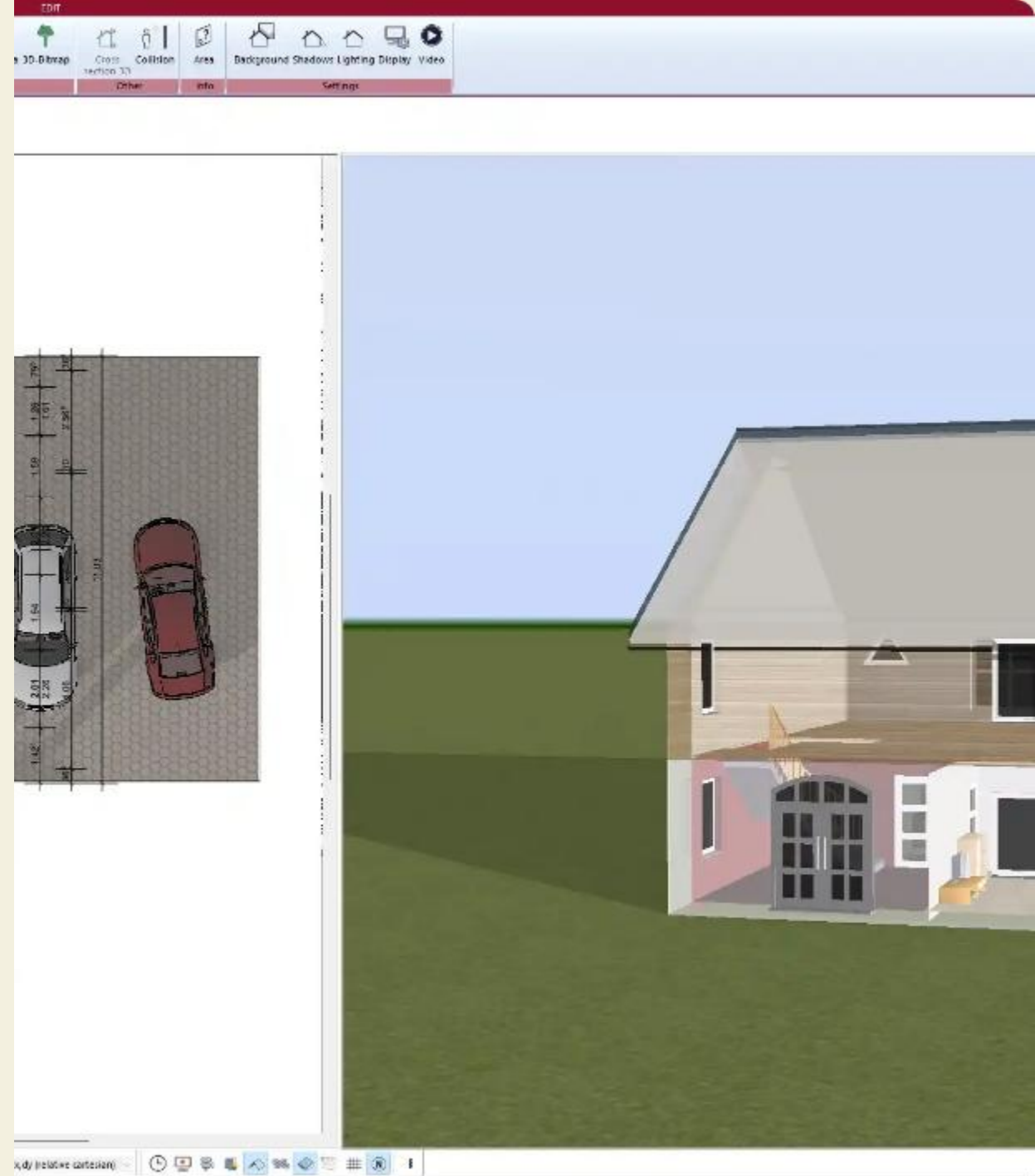
Conceito

O Molde (Blueprint)

Uma **Classe** é como se fosse a planta de uma casa. Ela não é a casa em si, mas define tudo o que a casa terá.

Ela define as **propriedades** (ex: cor, quantidade de portas, tamanho) e os **comportamentos** (ex: abrir porta, acender luz).

A partir de uma única planta (classe), podemos construir diversas casas.



2. Objetos

Conceito

A Instância

Se a classe é a planta, o **Objeto** é a casa construída e pronta para morar. Dizemos que o objeto é uma *instância* de uma classe.

Cada objeto criado pode ter seus próprios dados. Por exemplo, a planta (Classe Casa) diz que existe uma "cor". O objeto "Minha Casa" define que essa cor é azul.



Classe vs. Objeto



A Classe

Definição: É o conceito abstrato. A definição do tipo.

Exemplo Real: O conceito de "Cachorro". Todo cachorro tem nome, raça e late.

Memória: Não ocupa espaço na memória do programa (é apenas uma regra).



O Objeto

Definição: É a entidade concreta gerada a partir da classe.

Exemplo Real: O "Rex", que é um Poodle, e late bem alto.

Memória: Ocupa espaço real na memória (pois carrega dados físicos).

Os Pilares da POO

Encapsulamento, Herança e
Polimorfismo

3. Encapsulamento

Conceito

Protegendo o interior

É a prática de esconder os detalhes internos do funcionamento de um objeto, expondo apenas o que é seguro e necessário.

Exemplo: O Caixa Eletrônico. Você aperta botões na tela e o dinheiro sai. Você não precisa (e nem deve) ter acesso direto ao cofre ou às engrenagens da máquina para sacar dinheiro.



Vantagens do Encapsulamento



Segurança de Dados: Impede que variáveis importantes sejam modificadas indevidamente por outras partes do programa. O acesso é feito via métodos (getters/setters).



Simplicidade de Uso: O desenvolvedor que usa a classe não precisa entender a complexidade de como ela funciona por dentro, apenas como usá-la.



Facilidade de Manutenção: Podemos alterar o código interno da classe sem quebrar o resto do sistema que a utiliza.

4. Herança

Conceito

Passando características adiante

Permite que uma nova classe herde atributos e métodos de uma classe já existente. É como a genética familiar.

Uma classe **Filha** (subclasse) herda da classe **Mãe** (superclasse). Ela ganha tudo o que a mãe tem e ainda pode ter suas próprias características exclusivas.



Herança na Prática: Classe "Veículo"

Todos herdam características de **Veículo** (como acelerar, frear, chassi), mas possuem diferenças próprias.



Carro (4 portas, porta-malas)



Moto (Guidão, 2 rodas)



Caminhão (Carroceria, Carga máxima)

5. Polimorfismo

Conceito

Muitas Formas

Permite que objetos de classes diferentes respondam ao **mesmo comando** de maneiras diferentes.

Exemplo: O Controle Remoto. O botão "Ligar" faz uma TV ligar a tela, e um Ar-Condicionado ligar o motor. O comando (método) é o mesmo, mas a execução muda de acordo com o objeto.



Resumo dos Pilares



Encapsulamento

Esconde a complexidade e protege os dados internos da classe, garantindo a segurança e estabilidade do código.



Herança

Aproveita o código já criado, passando características de uma classe mais genérica para classes mais específicas.



Polimorfismo

Permite tratar diferentes objetos de forma padronizada, onde cada objeto sabe como executar a ação de forma particular.

Referência Bibliográfica

RAMALHO, Luciano. Python Fluente: Programação Clara, Concisa e Eficaz. 2. ed. São Paulo: Novatec, 2023.